Tutorial – 3

Code

```
"""
Tutorial 3: Statistical Foundations in Information Retrieval
Zipf's Law and Heap's Law
"""

import os
import re
import math
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from collections import Counter
import nltk

# Embedded Text Data Fallback (Thematic to IR)
# Contains multiple paragraphs of real text to ensure we have >= 2000 words.
EMBEDDED_TEXT = """
Information retrieval (IR) is the activity of obtaining information system resources that are relevant to an information need from a collection of those resources. Searches can be based on full-text or other content-based indexing. Information retrieval is the science of searching for information in a document, searching for documents themselves, and also searching for the metadata that describes data, and for databases of texts, images or sounds.
Automated information retrieval systems are used to reduce what has been called information overload. An IR system is a software system that provides access to books, journals and other documents; stores and manages those documents. Web search engines are the most visible IR applications.
""" * 50

def get_corpus():
    try:
        # Try to use a built-in NLTK collection
        nltk.download('gutenberg', quiet=True)
        from nltk.corpus import gutenberg
        # Get a substantial text: Alice in Wonderland
        text = gutenberg.raw('carroll-alice.txt')
        if len(text) > 10000:
            return text
    except:
        pass
    return EMBEDDED_TEXT

def clean_and_tokenize(text):
    text = text.lower()
    # Tokenize into words (only alphabetic tokens)
    tokens = re.findall(r'\b[a-z]+\b', text)
    return tokens

def part_a_zipfs_law(tokens, output_dir):
    print("\n--- Part A: Zipf's Law ---")

    # Count frequency
    word_counts = Counter(tokens)

    # Sort by frequency descending
    sorted_words = sorted(word_counts.items(), key=lambda x: x[1], reverse=True)

    # Assign rank
    ranked_data = []
    for rank, (word, freq) in enumerate(sorted_words, 1):
        ranked_data.append((rank, word, freq))

    # Display Top 20
    print(f"{'Rank':<10} | {'Word':<20} | {'Frequency':<10}")
    print("-" * 45)
    for rank, word, freq in ranked_data[:20]:
        print(f"{rank:<10} | {word:<20} | {freq:<10}")

    # Prepare data for plotting
    ranks = np.array([r for r, w, f in ranked_data])
    frequencies = np.array([f for r, w, f in ranked_data])

    # 1. Rank vs Frequency Plot
    plt.figure(figsize=(10, 6))
    plt.plot(ranks, frequencies, color='blue')
    plt.title("Zipf's Law: Rank vs. Frequency")
```

```
    plt.xlabel("Rank")
    plt.ylabel("Frequency")
    plt.grid(True)
    plt.savefig(os.path.join(output_dir, "zipfs_law_rank_vs_freq.png"))
    plt.close()

    # 2. Log(Rank) vs Log(Frequency) Plot
    log_ranks = np.log10(ranks)
    log_freqs = np.log10(frequencies)

    # Fit line
    slope, intercept = np.polyfit(log_ranks, log_freqs, 1)

    plt.figure(figsize=(10, 6))
    plt.plot(log_ranks, log_freqs, marker='.', linestyle='none', color='blue', label='Data')
    plt.plot(log_ranks, slope * log_ranks + intercept, color='red', label=f'Fitted Line (slope={slope:.2f})')
    plt.title("Zipf's Law: Log(Rank) vs. Log(Frequency)")
    plt.xlabel("Log10(Rank)")
    plt.ylabel("Log10(Frequency)")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(output_dir, "zipfs_law_log_vs_log.png"))
    plt.close()

    print(f"\nFitted Slope for Zipf's Law: {slope:.2f} (Expected ~ -1)")
    if -1.2 <= slope <= -0.8:
        print("Conclusion: The data strongly follows Zipf's Law.")
    else:
        print("Conclusion: The data loosely follows Zipf's Law.")

def part_b_heaps_law(tokens, output_dir):
    print("\n--- Part B: Heap's Law ---")

    total_words_list = []
    vocab_size_list = []

    current_vocab = set()
    step = len(tokens) // 15
    if step == 0: step = 100

    for i in range(step, len(tokens) + step, step):
        chunk = tokens[:i]
        current_vocab = set(chunk)
        total_words_list.append(len(chunk))
        vocab_size_list.append(len(current_vocab))

    print(f"{'Total Words (N)':<20} | {'Unique Words (V)':<20}")
    print("-" * 45)
    for n, v in zip(total_words_list, vocab_size_list):
        print(f"{n:<20} | {v:<20}")

    N = np.array(total_words_list)
    V = np.array(vocab_size_list)

    # 3. Total Words vs Vocabulary Size
    plt.figure(figsize=(10, 6))
    plt.plot(N, V, marker='o', color='green')
    plt.title("Heap's Law: Total Words vs. Vocabulary Size")
    plt.xlabel("Total Words (N)")
    plt.ylabel("Vocabulary Size (V)")
    plt.grid(True)
    plt.savefig(os.path.join(output_dir, "heaps_law_N_vs_V.png"))
    plt.close()

    # 4. Log(N) vs Log(V) Plot
    log_N = np.log10(N)
    log_V = np.log10(V)

    beta, log_K = np.polyfit(log_N, log_V, 1)
    K = 10 ** log_K

    plt.figure(figsize=(10, 6))
    plt.plot(log_N, log_V, marker='o', linestyle='none', color='green', label='Data')
    plt.plot(log_N, beta * log_N + log_K, color='orange', label=f'Fitted (β={beta:.2f}, K={K:.2f})')
    plt.title("Heap's Law: Log(N) vs. Log(V)")
    plt.xlabel("Log10(Total Words)")
    plt.ylabel("Log10(Vocabulary Size)")
    plt.legend()
    plt.grid(True)
    plt.savefig(os.path.join(output_dir, "heaps_law_log_vs_log.png"))
```

```
    plt.close()

    print(f"\nFitted Parameters: K = {K:.2f}, beta = {beta:.2f}")
    if 0.4 <= beta <= 0.6:
        print("Conclusion: The parameter beta is between 0.4 and 0.6, so the data follows Heap's Law.")
    else:
        print("Conclusion: The parameter beta is outside the standard 0.4-0.6 range. (Possible with very small or repetitive
text corpora)")

def main():
    script_dir = os.path.dirname(os.path.abspath(__file__))
    output_dir = script_dir

    print("Loading text corpus...")
    text = get_corpus()
    tokens = clean_and_tokenize(text)
    print(f"Total tokens collected: {len(tokens)}")

    part_a_zipfs_law(tokens, output_dir)
    part_b_heaps_law(tokens, output_dir)

    print(f"\nAll plots have been saved as PNGs in: {output_dir}")

if __name__ == '__main__':
    main()
```

Output

```
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials> python '.\Tutorial - 3\main.py'
Loading text corpus...
Total tokens collected: 27333

--- Part A: Zipf's Law ---
Rank       | Word                 | Frequency
-----------------------------------------
1          | the                  | 1642
2          | and                  | 872
3          | to                   | 729
4          | a                    | 632
5          | it                   | 595
6          | she                  | 553
7          | i                    | 543
8          | of                   | 514
9          | said                 | 462
10         | you                  | 411
11         | alice                | 398
12         | in                   | 369
13         | was                  | 357
14         | that                 | 315
15         | as                   | 263
16         | her                  | 248
17         | t                    | 218
18         | at                   | 212
19         | s                    | 201
20         | on                   | 193

Fitted Slope for Zipf's Law: -1.23 (Expected ~ -1)
Conclusion: The data loosely follows Zipf's Law.

--- Part B: Heap's Law ---
Total Words (N)      | Unique Words (V)
-----------------------------------------
1822                 | 543
3644                 | 819
5466                 | 1076
7288                 | 1249
9110                 | 1413
10932                | 1572
12754                | 1740
14576                | 1842
16398                | 1949
18220                | 2078
20042                | 2163
21864                | 2276
23686                | 2381
25508                | 2474
27330                | 2568
27333                | 2569

Fitted Parameters: K = 7.93, beta = 0.57
Conclusion: The parameter beta is between 0.4 and 0.6, so the data follows Heap's Law.

All plots have been saved as PNGs in: D:\SVNIT\Semester - 7\Information Retrieval\Tutorials\Tutorial - 3
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials>
```